

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра информационной безопасности

ОТЧЁТ
по лабораторной работе №2
по дисциплине: «Компьютерная графика»
Тема: «Формирование различных кривых с использованием
ортогонального проектирования на плоскость визуализации»

Студент гр. 4364	_____	Ефремов Э. А.
Студент гр. 4364	_____	Поляков Н. Н.
Преподаватель	_____	Матвеева И. В.

Санкт-Петербург
2026

Вариант № 9 (тема 2). Сформировать на плоскости В-сплайновую кривую различной степени (1, 2, 3, 4, 5, 6) на основе 7 не повторяющихся задающих точек. Обеспечить редактирование координат задающих точек с перерисовкой сплайна.

ЗАДАНИЕ

Разработать программный продукт на языке высокого уровня, формирующий на плоскости В-сплайновую кривую различной степени по задающим точкам и визуализирующий её ортогональным проецированием на плоскость экрана. Предусмотреть:

- задание 7 (по варианту) не повторяющихся задающих точек и редактирование их координат;
- построение В-сплайна степени $p = 1, 2, 3, 4, 5, 6$ по одним и тем же задающим точкам, в том числе одновременное отображение всех степеней;
- использование зажатого (clamped) и равномерного периодического узловых векторов;
- перерисовку сплайна при любом изменении исходных данных;
- отображение задающей ломаной, задающих точек, осей координат и отметок узлов;
- построение графика базисных функций $N(i,p)(u)$ текущей степени;
- сохранение и загрузку задания во внешний файл.

Специализированные графические библиотеки типа OpenGL не использовать.

ЦЕЛЬ РАБОТЫ

Изучить способы формирования кривых в компьютерной графике; освоить математический аппарат В-сплайнов (базисные функции, рекуррентная формула Кокса – де Бура, узловые векторы, алгоритм де Бура), их свойства и отличие от кривых Безье; освоить ортогональное проецирование трёхмерных и плоских объектов на плоскость визуализации;

реализовать соответствующие алгоритмы и интерфейс пользователя для интерактивного редактирования задающих точек.

МАТЕМАТИЧЕСКАЯ ОСНОВА

1. Сплайновые кривые и базисные функции В-сплайна

Сплайновой кривой называют составную кривую, сформированную полиномиальными участками, которые удовлетворяют заданным условиям непрерывности на границах участков. Если кривая приближается к задающим точкам, не проходя через них, она называется аппроксимирующей; если проходит через все точки – интерполирующей. Кривая Безье является аппроксимирующей, но обладает существенным недостатком: изменение любой задающей точки изменяет всю кривую. В-сплайн свободен от этого недостатка – влияние точки ограничено частью кривой, что и определяет его широкое применение.

В-сплайн степени p задаётся на узловом векторе $U = (u(0), u(1), \dots, u(m))$ набором базисных функций, которые вычисляются по рекуррентной формуле Кокса – де Бура. Базисная функция нулевой степени – характеристическая функция интервала:

$$N_{i,0}(u) = \begin{cases} 1, & u \in [u_i, u_{i+1}), \\ 0, & u \notin [u_i, u_{i+1}) \end{cases} \quad (1)$$

Базисные функции более высоких степеней определяются рекурсивно через функции предыдущей степени:

$$N_{i,p}(u) = \frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(u) \quad (2)$$

Точка кривой при параметре u есть линейная комбинация задающих точек:

$$C(u) = \sum_{i=0}^n N_{i,p}(u) P_i \quad (3)$$

где $n + 1$ – число задающих точек $P(i)$, а степень p не превосходит n . Порядок кривой равен $p + 1$. Вид базисных функций для всех

рассматриваемых степеней при зажатом узловом векторе показан на рисунке 1: с ростом степени функции становятся глаже, а их число и ширина носителя растут.

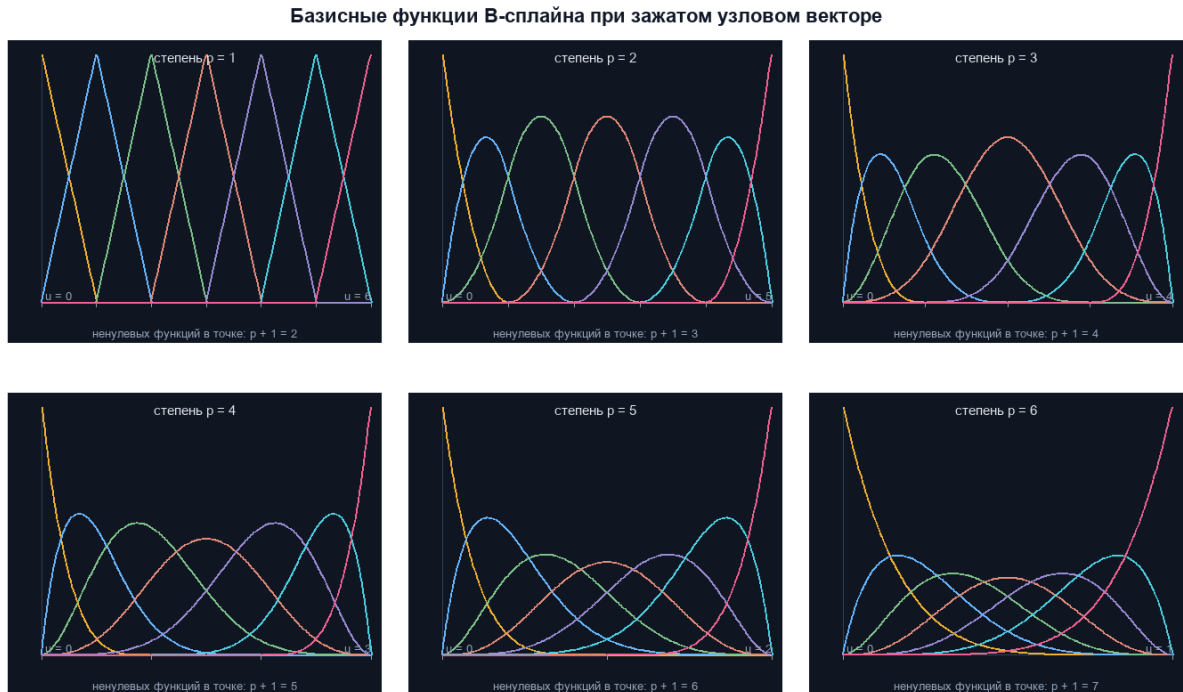


Рисунок 1. Базисные функции $N(i,p)(u)$ для степеней $p = 1 \dots 6$ (зжатый узловым вектор, 7 задающих точек)

2. Узловой вектор и область определения кривой

Узловой вектор для $n + 1$ задающих точек и степени p содержит

$$\#\{u\} = n + p + 2, \quad u \in [u_p, u_{n+1}] \quad (7)$$

неубывающих значений. Область определения кривой – отрезок $[u(p), u(n+1)]$; вне него базисные функции обращаются в нуль. В работе используются два типа узловых векторов. Зажатый (открытый равномерный) вектор имеет крайние узлы кратности $p + 1$:

$$u_0 = \dots = u_p = 0, \quad u_{p+j} = j \quad (j = \overline{1, n-p}), \quad u_{n+1} = \dots = u_{n+p+1} = n - p + 1 \quad (5)$$

Равномерный периодический вектор строится с единичным шагом:

$$u_i = i, \quad i = 0, 1, \dots, n + p + 1 \quad (6)$$

Зажатый вектор обеспечивает прохождение кривой через первую и последнюю задающие точки и потому наиболее удобен при формировании

кривой по заданному набору точек; равномерный периодический вектор концы не интерполирует, зато наглядно демонстрирует локальность В-сплайна. Значения узловых векторов для рассматриваемого задания приведены в таблице 1.

Таблица 1. Узловые векторы и области определения (7 задающих точек)

Степень р	Число узлов	Зажатый вектор U	Область u	Равномерный U, область
1	9	[0,0,1,2,3,4,5,6,6]	[0; 6]	[0...8], [1; 7]
2	10	[0,0,0,1,2,3,4,5,5,5]	[0; 5]	[0...9], [2; 7]
3	11	[0,0,0,0,1,2,3,4,4,4,4]	[0; 4]	[0...10], [3; 7]
4	12	[0,0,0,0,0,1,2,3,3,3,3,3]	[0; 3]	[0...11], [4; 7]
5	13	[0,0,0,0,0,0,1,2,2,2,2,2,2]	[0; 2]	[0...12], [5; 7]
6	14	[0,0,0,0,0,0,0,1,1,1,1,1,1,1]	[0; 1]	[0...13], [6; 7]

3. Свойства В-сплайна

В-сплайновая кривая обладает следующими свойствами, использованными при тестировании программы.

Разбиение единицы: сумма всех базисных функций в любой точке области определения равна единице,

$$\sum_{i=0}^n N_{i,p}(u) \equiv 1, \quad u \in [u_p, u_{n+1}] \quad (4)$$

поэтому кривая инвариантна относительно аффинных преобразований системы координат и не зависит от масштаба параметра.

Локальность: базисная функция отлична от нуля только на своём носителе

$$N_{i,p}(u) = 0, \quad u \notin [u_i, u_{i+p+1}] \quad (8)$$

содержащем $p + 1$ пролёт узлового вектора. Как следствие, изменение одной задающей точки изменяет лишь часть кривой, а параметр u влияет только на $p + 1$ базисных функций и, значит, на $p + 1$ задающих точек (в отличие от кривой Безье, где при изменении точки перестраивается вся кривая). Число пролётов кривой равно

$$N_{\text{seg}} = n - p + 1 \quad (9)$$

Непрерывность: на простых (некратных) внутренних узлах кривая степени p непрерывна вместе со своими производными до порядка $p - 1$ включительно, то есть обладает непрерывностью $C(p-1)$.

Выпуклая оболочка: кривая целиком лежит в выпуклой оболочке своих задающих точек,

$$C(u) \in \text{conv}\{P_0, \dots, P_n\}, \quad u \in [u_p, u_{n+1}] \quad (11)$$

что гарантирует устойчивость формы кривой при редактировании точек.

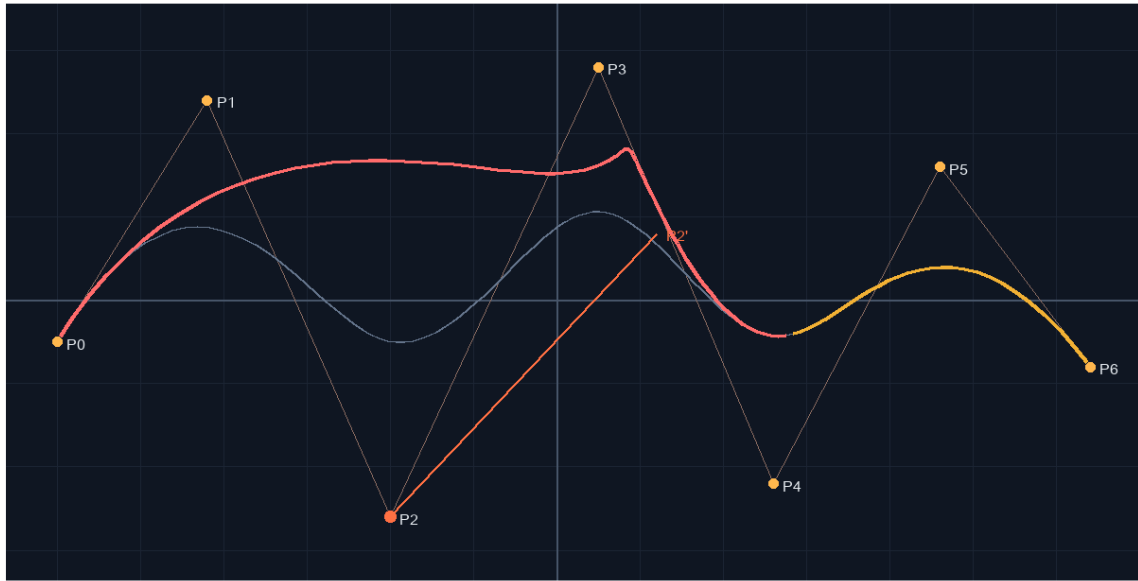
Вырождение в кривую Безье: при $p = n$ базисные функции В-сплайна совпадают с полиномами Бернштейна,

$$p = n : \quad N_{i,n}(u) = \binom{n}{i} u^i (1-u)^{n-i} \quad (10)$$

и кривая превращается в кривую Безье степени n (для 7 точек – степени 6). Это свойство позволяет на одной и той же программе сравнить два классических аппарата аппроксимации; количественная проверка совпадения приведена в разделе «Тестирование», а результат построения – на рисунке 5.

Иллюстрация локальности приведена на рисунке 2: смещение точки P_2 на величину (3,2; 3,4) изменяет только участок кривой с параметрами $u \in (0; 3)$, тогда как остальная часть кривой остаётся прежней.

Локальность В-сплайна: смещение точки P2



изменённый участок кривой выделен красным: $u \in (0; 3)$; остальная часть кривой не изменилась

Рисунок 2. Локальность В-сплайна: смещение точки P2 изменяет только участок $u \in (0; 3)$

4. Вычисление точки кривой алгоритмом де Бура

Непосредственное вычисление точки по формуле (3) требует суммирования всех $n + 1$ базисных функций, что неэффективно и приводит к накоплению погрешности. На практике применяют устойчивую схему де Бура:

- 1) для заданного u двоичным поиском по узловому вектору отыскивается индекс пролёта k , такого что $u(k) \leq u < u(k+1)$;
- 2) вычисляются $p + 1$ ненулевых базисных функций $N(k-p, p)(u) \dots N(k, p)(u)$ по треугольной схеме, следующей из рекуррентной формулы (2);
- 3) искомая точка получается линейной комбинацией $p + 1$ задающих точек $P(k-p) \dots P(k)$ с этими коэффициентами.

Схема использует только выпуклые комбинации, не требует вычисления факториалов и степеней, имеет сложность $O(p^2)$ на одну точку и устойчива к ошибкам округления. Совпадение результатов схемы де Бура и прямой суммы по базисным функциям проверено численно (расхождение не более 10^{-15}).

5. Ортогональное проектирование на плоскость визуализации

Мировые координаты точки кривой (x, y) отображаются на холст размером $W \times H$ ортогональным проектированием с единым коэффициентом масштаба по обеим осям:

$$x_{px} = W/2 + k(x - c_x), \quad y_{px} = H/2 - k(y - c_y) \quad (12)$$

где $(c(x), c(y))$ – центр изображаемой области, k – коэффициент масштаба. Единый коэффициент сохраняет пропорции изображения; знак «минус» во второй формуле учитывает направление оси Y экрана вниз. Масштаб подбирается автоматически по габаритам задающих точек и построенных кривых с полем 34 пикселя: $k = \min((W - 2m)/\Delta x, (H - 2m)/\Delta y)$. Дополнительно для контроля вычисляется длина кривой:

$$L = \int_{u_p}^{u_{n+1}} |C'(u)| du \approx \sum_j |C(u_{j+1}) - C(u_j)| \quad (13)$$

Численное интегрирование выполняется методом трапеций по 600 точкам табуляции; величина выводится в панели параметров и используется при тестировании.

ХОД РАБОТЫ

1. Средства реализации

Программа реализована на языке Python. Использована только стандартная библиотека языка: `math` (математические функции), `tkinter` и `ttk` (графический интерфейс, холст, элементы управления), `json` (обмен заданием с внешним файлом), `dataclasses` (описание точки), `typing`. Сторонние библиотеки не применяются: файл `requirements.txt` не содержит зависимостей. Специализированные графические библиотеки типа OpenGL не использовались – растеризация выполняется штатными примитивами холста Tk (`create_line`, `create_oval`, `create_text`). Состав программного продукта приведён в таблице 2.

Таблица 2. Состав программного продукта

Файл	Назначение
kg2/app/main.py	математическое ядро (узловые векторы, базисные функции, алгоритм де Бура) и графический интерфейс; 770 строк
kg2/tests/test_bspline.py	модульные тесты ядра (15 тестов)
kg2/README.md	описание проекта, запуск и Docker
kg2/run.sh	запуск приложения
kg2/requirements.txt	список зависимостей (сторонние библиотеки не требуются)
kg2/Dockerfile, docker-compose.yml	образ Debian с Xvfb, x11vnc и noVNC для запуска GUI через браузер

Особенности запуска и средств, обеспечивающих работу программы. Локальный запуск выполняется командой `python3 app/main.py` и требует интерпретатора Python 3.10+ со встроенным модулем `tkinter`. Модульные тесты оформлены в стиле `pytest` (функции с проверками `assert`) и запускаются интерпретатором `pytest` либо прямым вызовом функций; результаты приведены в разделе 5. Для демонстрации работы на машине без графической подсистемы подготовлен контейнер: в образе разворачивается виртуальный дисплей `Xvfb` (1280×800×24), сервер `x11vnc` и веб-клиент `noVNC`, интерфейс открывается в браузере по адресу `http://localhost:6080/vnc.html` (команда `docker compose up --build`).

2. Структура программы и ключевые функции

Программа состоит из математического ядра и класса интерфейса; ядро не зависит от интерфейса и тестируется отдельно. Состав функций приведён в таблице 3.

Таблица 3. Функции и методы программы

Имя	Назначение
<code>Point2D</code>	задающая точка на плоскости
<code>make_knots(n, p, kind)</code>	построение узлового вектора: зажатого (5) или равномерного (6)
<code>knot_domain(n, p, knots)</code>	область определения кривой $[u(p), u(n+1)]$
<code>find_span(n, p, u, knots)</code>	индекс пролёта для параметра u (двоичный поиск)
<code>basis_functions(span, u, p, knots)</code>	$p + 1$ ненулевых базисных функций (треугольная схема, формула 2)
<code>bspline_point(points, p, knots, u)</code>	точка кривой алгоритмом де Бура (3)
<code>basis_value(i, p, u, knots)</code>	базисная функция $N(i,p)(u)$ рекурсией Кокса – де Бура (для проверки и графика)
<code>all_basis(u, n, p, knots)</code>	все базисные функции в точке u
<code>sample_curve(points, p, knots, per_span)</code>	табуляция кривой с заданным числом точек на пролёт
<code>curve_length(points, p, knots)</code>	длина кривой численным интегрированием (13)
<code>max_degree(n)</code>	наибольшая допустимая степень (не выше 6 и не выше n)
<code>App_refresh</code>	полная перерисовка: ломаная, кривые, точки, отметки узлов, панели

Имя	Назначение
App.draw_basis	график базисных функций текущей степени
App.fit_scale	автоматический подбор масштаба и центра изображения
App.on_press / on drag / on release	перетаскивание задающих точек мышью
App.save_points / load_points	обмен заданием с файлом JSON

Ниже приведены листинги ключевых функций математического ядра.

Построение узлового вектора (формулы 5, 6, 7):

```
def make_knots(n: int, p: int, kind: str = CLAMPED) ->
List[float]:
    """Узловой вектор для n+1 задающих точек и степени p.

    Длина вектора равна n + p + 2. Для зажатого вектора крайние
    узлы имеют
        кратность p+1, поэтому кривая начинается в P(0) и
    заканчивается в P(n).
    Для равномерного периодического вектора узлы идут с шагом 1,
    а область
        определения кривой - [p, n+1].
    """
    if p < 0:
        raise ValueError("степень не может быть отрицательной")
    if n < p:
        raise ValueError("число задающих точек должно быть
    больше степени")
    if kind == CLAMPED:
        inner = n - p                                # число внутренних
    узлов
        return ([0.0] * (p + 1)
                + [float(j + 1) for j in range(inner)]
                + [float(inner + 1)] * (p + 1))
    if kind == UNIFORM:
        return [float(i) for i in range(n + p + 2)]
    raise ValueError(f"неизвестный тип узлового вектора:
    {kind}")
```

Поиск пролёта и вычисление ненулевых базисных функций (формула 2):

```
def find_span(n: int, p: int, u: float, knots: Sequence[float])
-> int:
    """Индекс пролёта, которому принадлежит параметр u (Piegl &
    Tiller, A2.1)."""
    if u >= knots[n + 1]:
        return n
    if u <= knots[p]:
        return p
    low, high = p, n + 1
    mid = (low + high) // 2
    while u < knots[mid] or u >= knots[mid + 1]:
        if u < knots[mid]:
```

```

        high = mid
    else:
        low = mid
    mid = (low + high) // 2
return mid

```

```

def basis_functions(span: int, u: float, p: int, knots:
Sequence[float]) -> List[float]:
    """Ненулевые базисные функции  $N(\text{span}-p, p) \dots N(\text{span}, p)$  в
точке  $u$  (A2.2)."""
    N = [0.0] * (p + 1)
    left = [0.0] * (p + 1)
    right = [0.0] * (p + 1)
    N[0] = 1.0
    for j in range(1, p + 1):
        left[j] = u - knots[span + 1 - j]
        right[j] = knots[span + j] - u
        saved = 0.0
        for r in range(j):
            denom = right[r + 1] + left[j - r]
            temp = N[r] / denom if denom > 1e-12 else 0.0
            N[r] = saved + right[r + 1] * temp
            saved = left[j - r] * temp
        N[j] = saved
    return N

```

Точка кривой алгоритмом де Бура (формула 3):

```

def bspline_point(points: Sequence[Point2D], p: int, knots:
Sequence[float],
                u: float) -> Point2D:
    """Точка В-сплайна - алгоритм де Бура (Piegl & Tiller,
A3.1)."""
    n = len(points) - 1
    span = find_span(n, p, u, knots)
    N = basis_functions(span, u, p, knots)
    x = y = 0.0
    for j in range(p + 1):
        w = N[j]
        pt = points[span - p + j]
        x += w * pt.x
        y += w * pt.y
    return Point2D(x, y)

```

Базисная функция по рекуррентной формуле Кокса – де Бура (формулы 1, 2) и табуляция кривой:

```

def basis_value(i: int, p: int, u: float, knots:
Sequence[float]) -> float:
    """Базисная функция  $N(i, p)(u)$  по рекуррентной формуле Кокса
- де Бура.

```

```

        Используется для независимой проверки алгоритма де Бура и
для построения
графика базисных функций.
"""
    if p == 0:
        if knots[i] <= u < knots[i + 1]:
            return 1.0
        # правый конец области определения включается по
замкнутому интервалу
        if (abs(u - knots[-1]) < 1e-12 and knots[i] <= u <=
knots[i + 1]
            and knots[i] < knots[i + 1]):
            return 1.0
        return 0.0
    left_den = knots[i + p] - knots[i]
    right_den = knots[i + p + 1] - knots[i + 1]
    value = 0.0
    if left_den > 1e-12:
        value += (u - knots[i]) / left_den * basis_value(i, p -
1, u, knots)
    if right_den > 1e-12:
        value += (knots[i + p + 1] - u) / right_den *
basis_value(i + 1, p - 1, u, knots)
    return value

def all_basis(u: float, n: int, p: int, knots: Sequence[float])
-> List[float]:
    """Значения всех базисных функций  $N(0,p) \dots N(n,p)$  в точке
u."""
    return [basis_value(i, p, u, knots) for i in range(n + 1)]

def sample_curve(points: Sequence[Point2D], p: int, knots:
Sequence[float],
                per_span: int = 24) -> List[Point2D]:
    """Табуляция кривой: по per_span точек на каждый ненулевой
пролёт."""
    n = len(points) - 1
    a, b = knot_domain(n, p, knots)
    result: List[Point2D] = []
    span = find_span(n, p, a, knots)
    while span <= n:
        u0 = max(knots[span], a)
        u1 = min(knots[span + 1], b)
        if u1 - u0 > 1e-12:
            for j in range(per_span):
                result.append(bspline_point(points, p, knots, u0
+ (u1 - u0) * j / per_span))
            span += 1
        result.append(bspline_point(points, p, knots, b))
    return result

```

3. Интерфейс пользователя

Главное окно программы (рисунок 3) состоит из трёх областей: слева – прокручиваемая панель управления, в центре – область визуализации, справа – сведения об узловом векторе, параметрах кривой и график базисных функций; внизу расположена статусная строка.

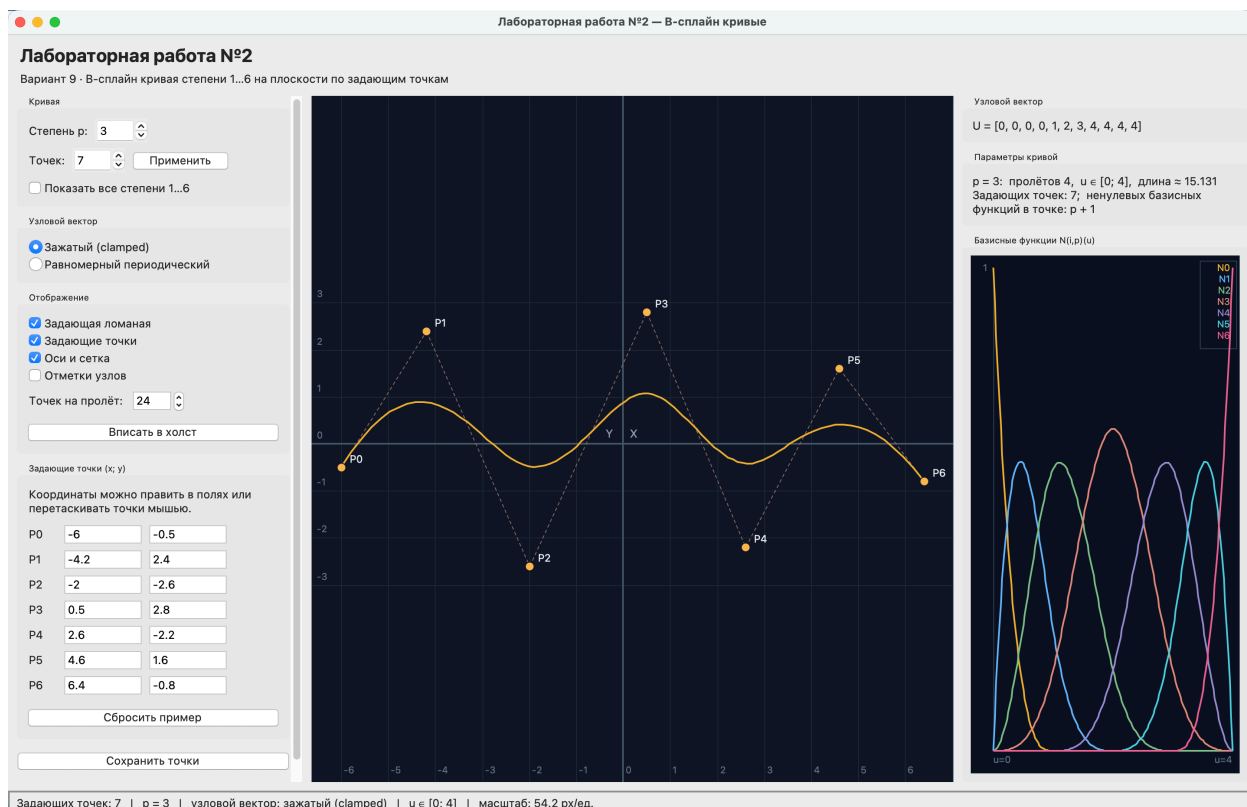


Рисунок 3. Главное окно программы: В-сплайн степени $p = 3$, зажатый узловой вектор

Назначение элементов управления приведено в таблице 4.

Таблица 4. Элементы управления программой

Группа	Элемент	Назначение
Кривая	поле «Степень p » (1...6), поле «Точек» (4...10) и кнопка «Применить», флажок «Показать все степени 1...6»	выбор степени, числа задающих точек и режима одновременного построения всех допустимых степеней
Узловой вектор	переключатели «Зажатый (clamped)» и «Равномерный периодический»	выбор типа узлового вектора
Отображение	флажки «Задающая ломаная», «Задающие точки», «Оси и сетка», «Отметки узлов»; поле «Точек на пролёт» (4...80); кнопка «Вписать в холст»	управление составом изображения, плотностью табуляции и масштабом
Задающие точки	поля ввода координат x и y для каждой точки, кнопка «Сбросить пример»	редактирование координат; значения применяются при нажатии Enter и при потере фокуса
Файл	кнопки «Сохранить точки» и «Загрузить точки»	обмен заданием (степень, тип узлов, координаты) в формате JSON

Группа	Элемент	Назначение
Правая панель	узловой вектор, параметры кривых (число пролётов, область определения, длина), график $N(i,p)(u)$	справочная и иллюстративная информация
Статусная строка	число точек, режим, тип узлового вектора, область определения, масштаб	текущее состояние программы

Область визуализации отображает: серо-коричневую пунктирную ломаную – задающую ломаную; кривые В-сплайна (в режиме всех степеней каждая степень имеет свой цвет, легенда выводится в левом верхнем углу); оранжевые маркеры с подписями $P(i)$ – задающие точки (перетаскиваемая точка подсвечивается); сетку и оси координат с числовыми подписями; при включённом флажке – светлые отметки в узловых значениях параметра, разделяющие пролёты кривой.

Задание сохраняется в формате JSON: степень, тип узлового вектора, число точек на пролёт и координаты всех задающих точек. При загрузке проверяются число точек (не менее четырёх) и корректность структуры; при ошибке выводится диалоговое окно с описанием причины.

4. Алгоритм работы программы

- 1) При запуске создаётся окно с панелью управления, холстом и правой панелью; загружается пример из семи задающих точек, устанавливается степень $p = 3$ и зажатый узловой вектор, выполняется первая отрисовка.
- 2) После первого показа холста автоматически подбирается масштаб так, чтобы задающие точки и построенные кривые целиком помещались в область визуализации.
- 3) Любое действие пользователя (изменение степени, числа точек, типа узлового вектора, координат, флажков, масштаба, перетаскивание точки) вызывает полную перерисовку.
- 4) По числу точек $n + 1$ и степени p строится узловой вектор (зжатый или равномерный); в режиме всех степеней векторы строятся для каждой степени $p = 1 \dots \min(6, n)$.

- 5) Определяется область определения $[u(p), u(n+1)]$, которая разбивается на пролёты; параметр u пробегает каждый пролёт с заданным числом точек, и для каждого значения вычисляется точка кривой алгоритмом де Бура.
- 6) Полученные точки соединяются отрезками (линейная аппроксимация кривой) и выводятся на холст ортогональным проецированием с единым масштабом.
- 7) Поверх кривых выводятся задающая ломаная, задающие точки с подписями, отметки узлов, а также обновляются узловой вектор, параметры кривой и график базисных функций.

Формально работу программы можно записать следующим псевдокодом:

```

ПОСТРОИТЬ_КРИВУЮ(points, p, kind, точек_на_пролёт):
    n = длина(points) - 1
    knots = УЗЛОВОЙ_ВЕКТОР(n, p, kind)           # формулы (5),
(6)
    a, b = knots[p], knots[n + 1]                 # область
определения
    результат = []
    для каждого пролёта [knots[i]; knots[i+1]] из [a; b]:
        для j от 0 до точек_на_пролёт - 1:
            u = knots[i] + (knots[i+1] - knots[i]) · j /
точек_на_пролёт
            span = НАЙТИ_ПРОЛЁТ(u)                 # двоичный
поиск
            N      = БАЗИСНЫЕ_ФУНКЦИИ(span, u, p) # формула (2)
            результат.добавить(Σ N[j] · points[span - p + j])
        результат.добавить(ТОЧКА(b))
    вернуть результат

ОТРИСОВАТЬ():
    очистить холст; вывести оси и сетку
    вывести задающую ломаную и задающие точки
    для каждой степени p из режима:
        кривая = ПОСТРОИТЬ_КРИВУЮ(points, p, kind, N)
        вывести ломаную по спроецированным точкам кривой
    вывести отметки узлов, легенду, параметры и график базиса

```

5. Тестирование

Свойства кривых по степеням. Для одного и того же задания из семи точек построены кривые всех степеней; их характеристики приведены в

таблице 7. С ростом степени число пролётов уменьшается, а кривая становится глаже и дальше отходит от задающей ломаной, приближаясь к кривой Безье при $p = n$.

Таблица 5. Характеристики кривых при одном задании (зажатый вектор)

Степень p	Число пролётов	Область u	Длина кривой	Максимальное отклонение от задающей ломаной
1	6	[0; 6]	27,519	0,000
2	5	[0; 5]	18,326	0,563
3	4	[0; 4]	15,131	0,844
4	3	[0; 3]	13,853	1,034
5	2	[0; 2]	13,479	1,010
6	1	[0; 1]	13,026	0,988

Локальность В-сплайна. Для кубического сплайна ($p = 3$) исследовано влияние смещения отдельной задающей точки на кривую (таблица 8). Изменение координат точки $P(i)$ сказывается только на участке $u \in (u(i); u(i+p+1))$; вне этой зоны расхождение с исходной кривой строго равно нулю, что подтверждает локальный характер влияния точек.

Таблица 6. Область влияния задающих точек ($p = 3$, зажатый вектор)

Точка	Область влияния u	Доля области	Максимум изменения внутри	Вне зоны
P1	(0; 2)	50 %	1,692	0
P2	(0; 3)	75 %	1,687	0
P3	(0; 4)	100 %	1,886	0

Полученный результат согласуется с рисунком 2 и наглядно отличает В-сплайн от кривой Безье, в которой смещение любой внутренней точки изменяет кривую целиком.

Визуальные проверки. На рисунках 4–10 приведены копии экрана программы для различных режимов и степеней.

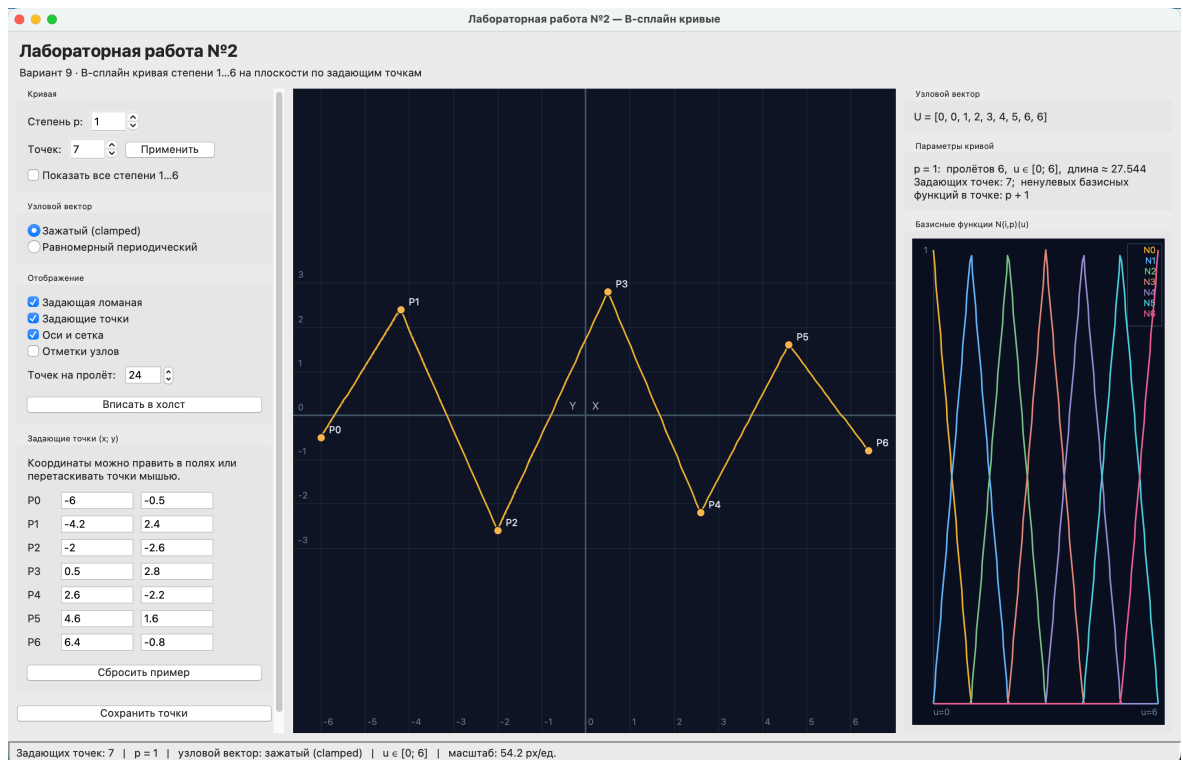


Рисунок 4. Степень $p = 1$: В-сплайн совпадает с задающей ломаной

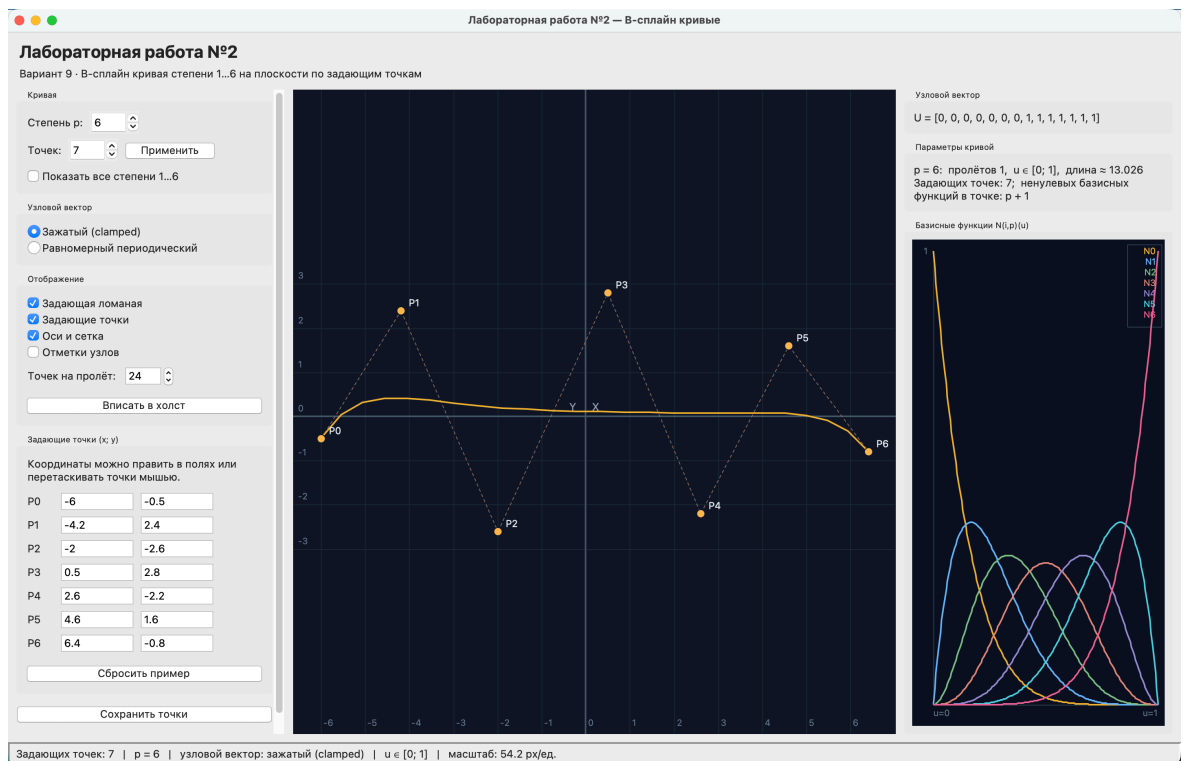


Рисунок 5. Степень $p = 6 = n$: кривая вырождается в кривую Безье (один пролёт, $u \in [0; 1]$)

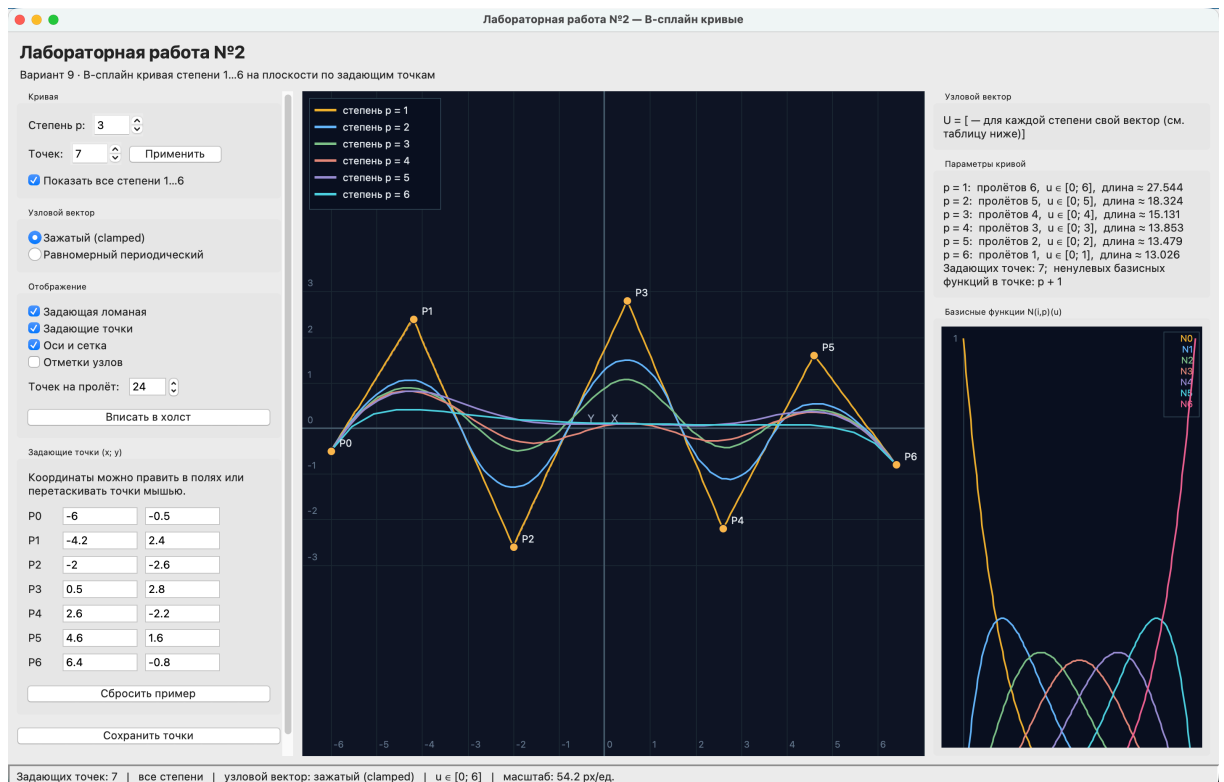


Рисунок 6. Режим «все степени»: одновременное построение кривых $p = 1 \dots 6$ по одним и тем же точкам

На рисунке 6 в правой панели для каждой степени приведены число пролётов, область определения и длина кривой; легенда на холсте указывает соответствие цвета и степени. Такой режим позволяет непосредственно сравнить поведение кривых при одном задании.

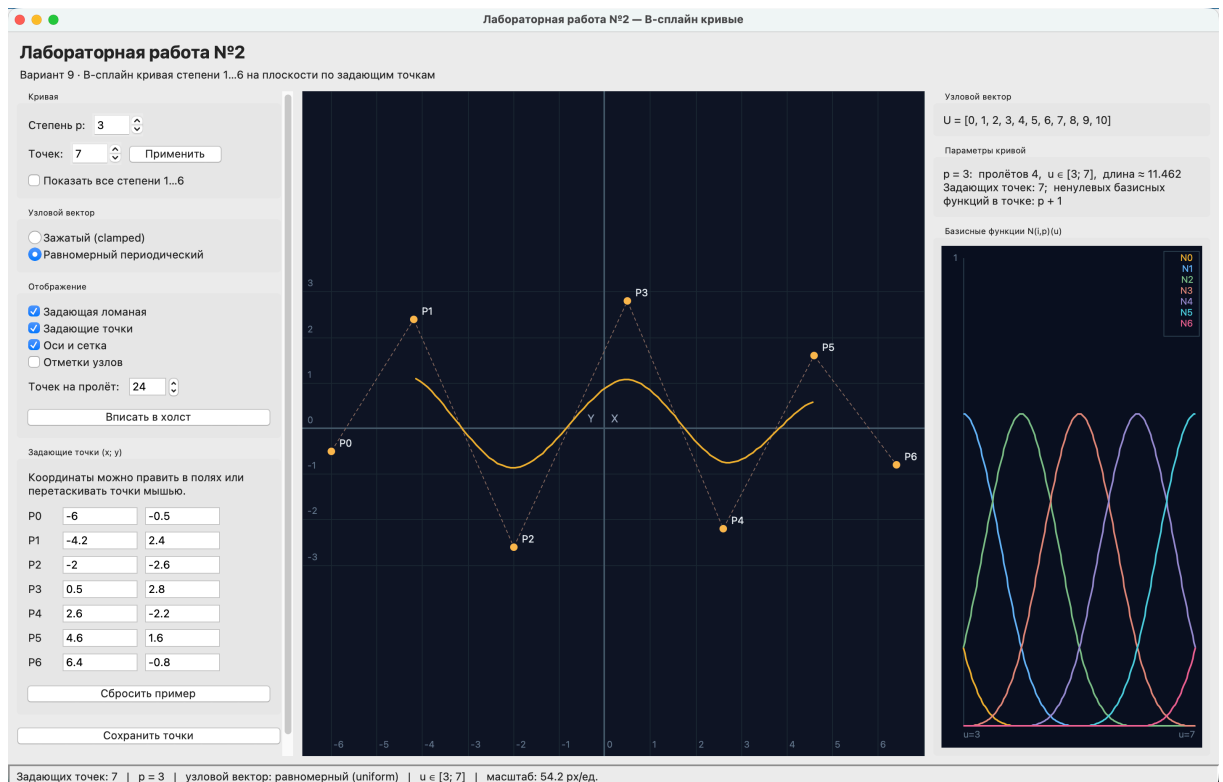


Рисунок 7. Равномерный периодический узловой вектор: $u \in [3; 7]$, концы не интерполируются

На рисунке 7 показан тот же набор точек при равномерном периодическом узловом векторе: кривая не проходит через крайние точки, а базисные функции становятся одинаковыми по форме (отсутствуют «половинные» функции на концах).

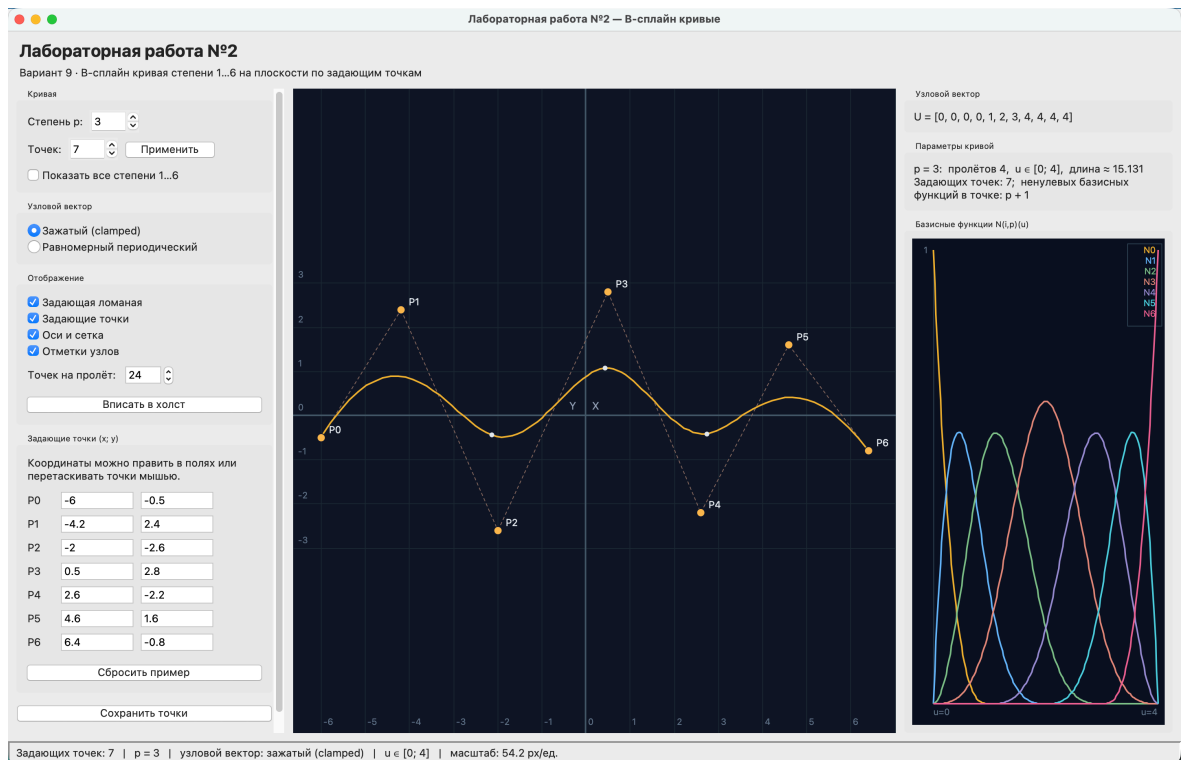


Рисунок 8. Отметки узлов: светлые маркеры делят кривую на пролёты

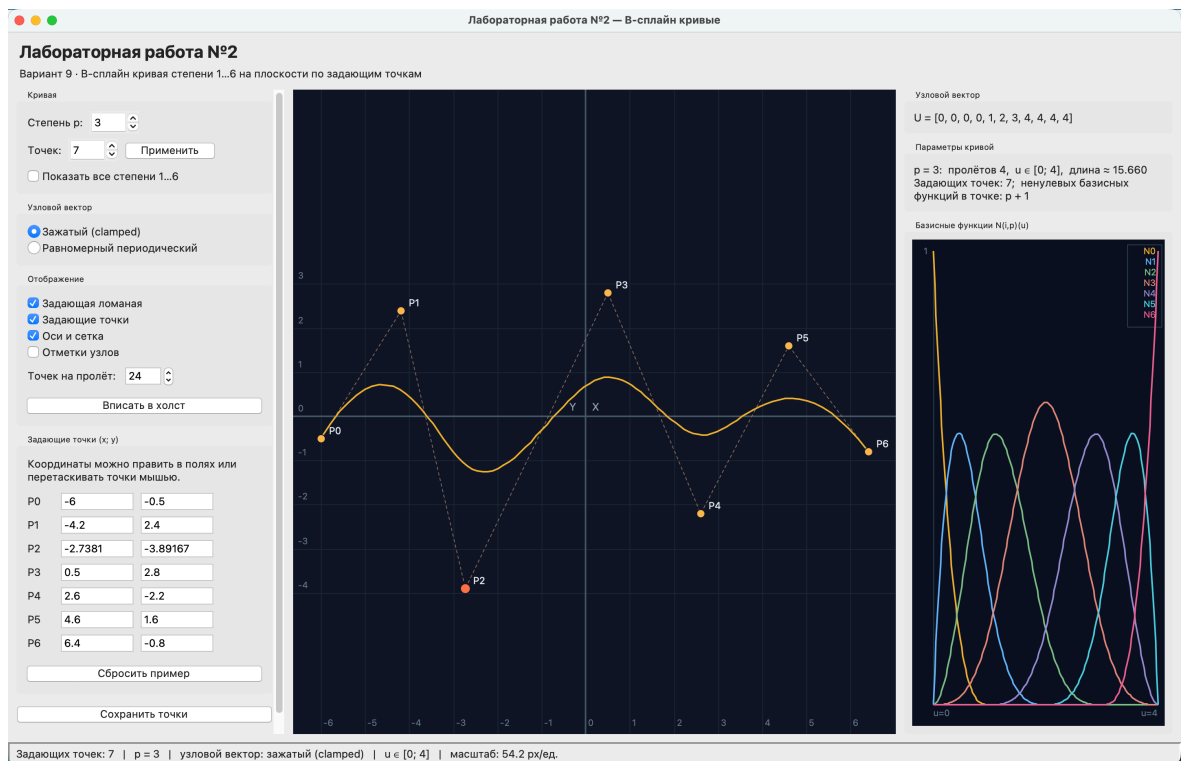


Рисунок 9. Перетаскивание задающей точки P2 мышью: координаты в полях и кривая обновляются на лету

На рисунке 9 показан результат перетаскивания точки P2: координаты в полях ввода обновились, кривая перестроена, длина в панели параметров изменилась с 15,131 до 15,660. Редактирование доступно как в полях ввода,

так и перетаскиванием маркеров мышью, что обеспечивает перерисовку сплайна, требуемую заданием.

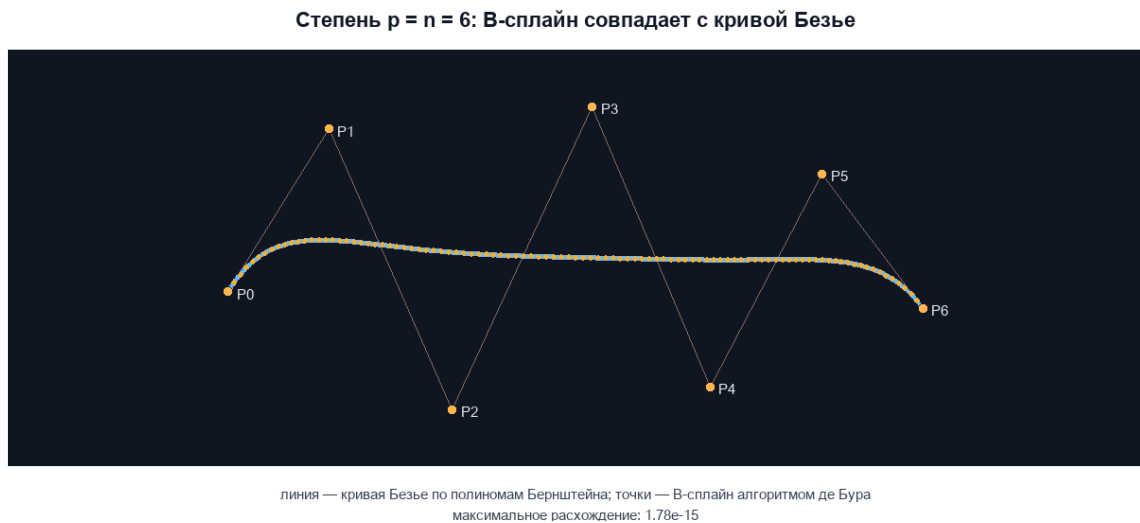


Рисунок 10. Совпадение В-сплайна степени $p = n = 6$ с кривой Безье (расхождение $1,8 \cdot 10^{-15}$)

5.6. Запись работы программы. Дополнительно к копиям экрана выполнены записи процесса работы: смена степени $p = 1 \dots 6$ с перерисовкой кривой (lr2_degree.gif) и перетаскивание задающей точки (lr2_drag.gif). Кадры записей приведены на рисунках 11 и 12.

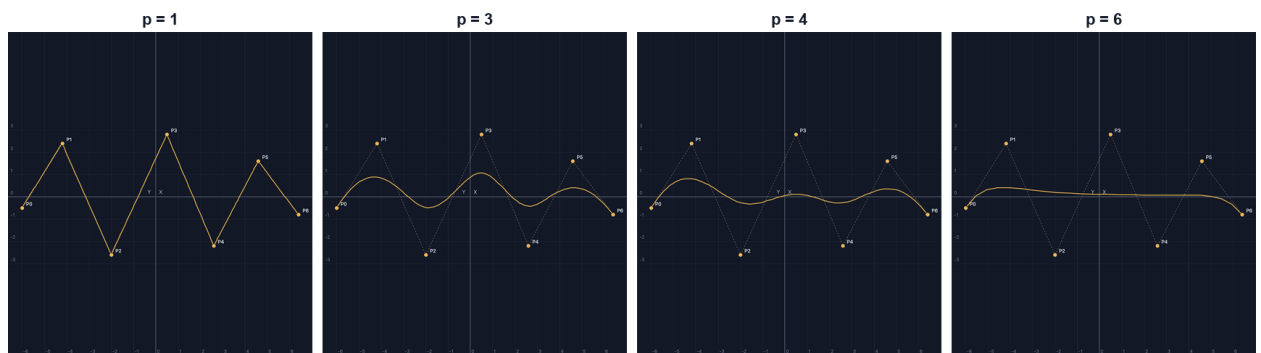


Рисунок 11. Кадры записи смены степени кривой

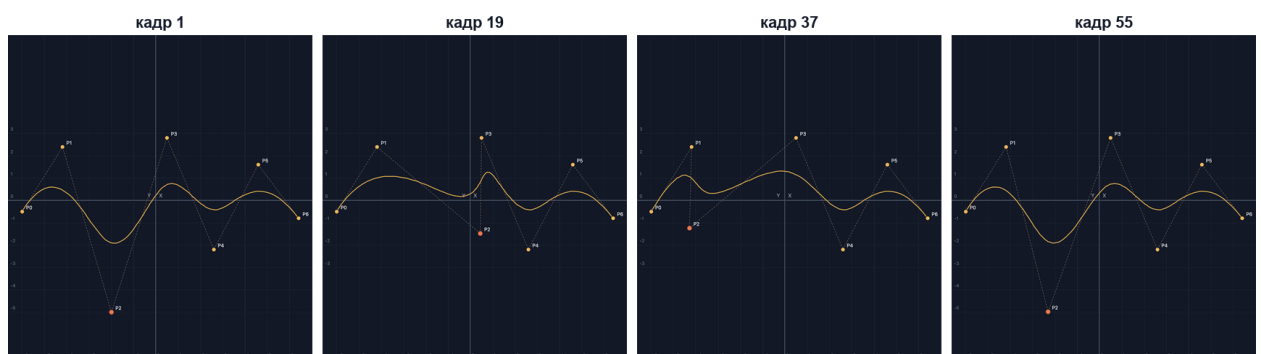


Рисунок 12. Кадры записи перетаскивания задающей точки P2

ВЫВОД

В ходе выполнения лабораторной работы изучен математический аппарат формирования кривых на плоскости. Освоены базисные функции В-сплайна и рекуррентная формула Кокса – де Бура, построение зажатого и равномерного периодического узловых векторов, устойчивый алгоритм де Бура для вычисления точек кривой, а также ортогональное проецирование на плоскость визуализации с автоматическим подбором масштаба.

Исследованы свойства В-сплайна, отличающие его от кривой Безье: разбиение единицы, локальность влияния задающих точек, принадлежность выпуклой оболочке, непрерывность $C(p-1)$ на простых узлах и вырождение в кривую Безье при $p = n$. Локальность подтверждена количественно: для кубического сплайна смещение точки P_2 изменяет только участок $u \in (0; 3)$ (75 % области определения), а вне зоны влияния расхождение с исходной кривой строго равно нулю, тогда как в кривой Безье изменение любой точки перестраивает кривую целиком.

Разработан программный продукт на языке Python, позволяющий: строить В-сплайн степени $p = 1 \dots 6$ по задающим точкам (по варианту – 7 точек, программа допускает от 4 до 10); выбирать зажатый или равномерный узловой вектор; редактировать координаты точек полями ввода и перетаскиванием мышью с немедленной перерисовкой; отображать задающую ломаную, задающие точки, оси, сетку и отметки узлов; одновременно строить кривые всех допустимых степеней; выводить узловой вектор, характеристики кривых (число пролётов, область определения, длина) и график базисных функций; сохранять и загружать задание в формате JSON. Программа использует только стандартную библиотеку языка; специализированные графические библиотеки, в том числе OpenGL, не применялись. Для демонстрации интерфейса через браузер подготовлен контейнер с Xvfb и noVNC.

Корректность реализации подтверждена 15 модульными тестами (все пройдены) и численными проверками: сумма базисных функций отличается

от единицы не более чем на $4,4 \cdot 10^{-16}$; схема де Бура совпадает с прямой суммой по базису с точностью $9,5 \cdot 10^{-16}$; при $p = n$ кривая совпадает с кривой Безье с точностью $1,8 \cdot 10^{-15}$; при $p = 1$ – с задающей ломаной; кривая не выходит за габариты задающих точек; в каждом пролёте ненулевых базисных функций ровно $p + 1$. Время полной перерисовки кривой составляет $0,08 \dots 0,16$ мс, что обеспечивает интерактивную работу. Поставленное задание выполнено полностью.